parallel merge sort 03, (Binary Search Based); Parallel sorting a Tile of array!

Discussion on mergeSortSharedKernel( )

Input:

* d_Dstkey: sorted keys array
* d_DstVal: sorted values associated with each key.

This is useful when sort large records,
The Basic Idea: when you move keys around, you also move the value into corresponding position.
If you store the Index number of each record before sorting into d_SrcVal.
After Sorted, d_DstVal will be a sorted Index array!

d_Srckey	7	2	0	5
d_SrcVal	0	1	2	3

d_Dstkey	0	2	5	7
d_DstVal	2	1	3	0

you can used this Sorted Index to access Other fields in the Input record array!

Very Useful when you do KNN nearest Neighbor Algorithm!

parallel merge sort 03 : (Binary Search Based)
Focus on kernel (parallel sorting a tile)

Notice : d_DstKey
d_DstVal ... } Input size could be much much bigger than the size of a tile. The kernel sorts a tile (a piece) within this input array.
Each Thread Block sorts a different tile in input array !

Illustration :

① we get into kernel code :

Assume here : SHARED_SIZE_LIMIT is 8 ;
This is the number of elements each block processes.
Actually we need only 4 threads to merge 8 elements ;

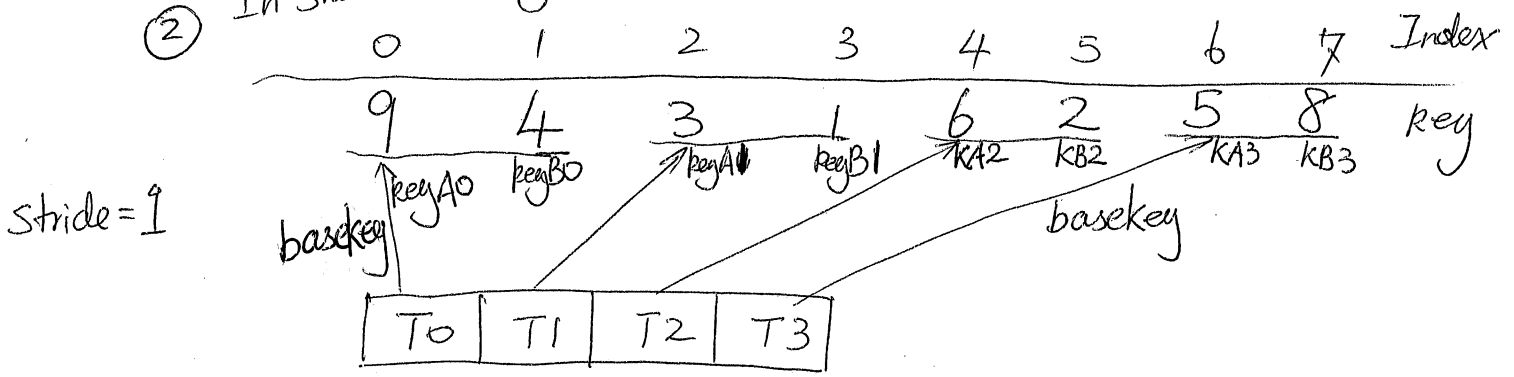
→ So size of thread block is
SHARED_SIZE_LIMIT / 2 ;

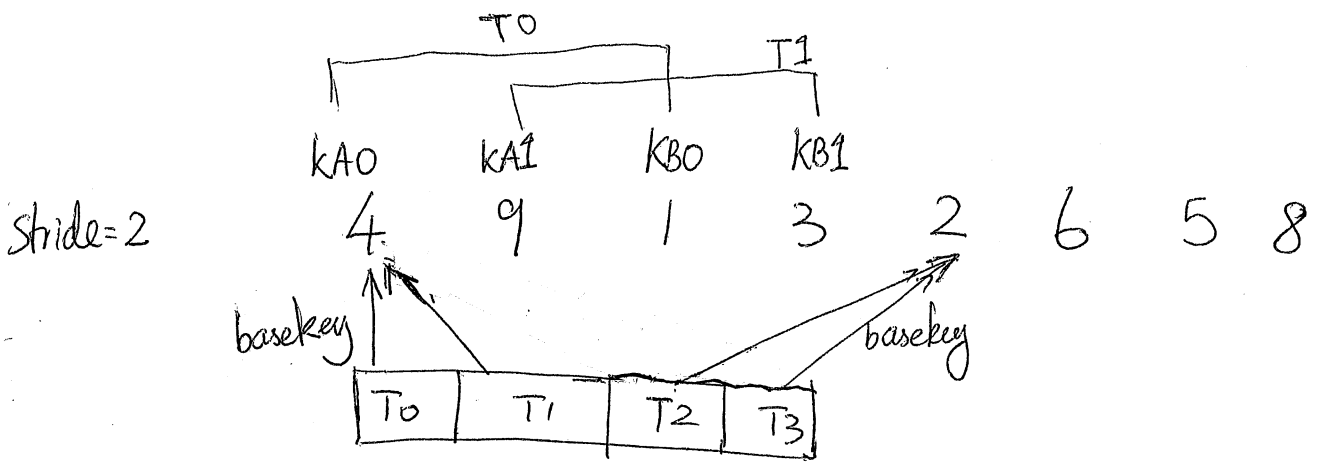
→ That's why each thread moves two elements, and process two elements.

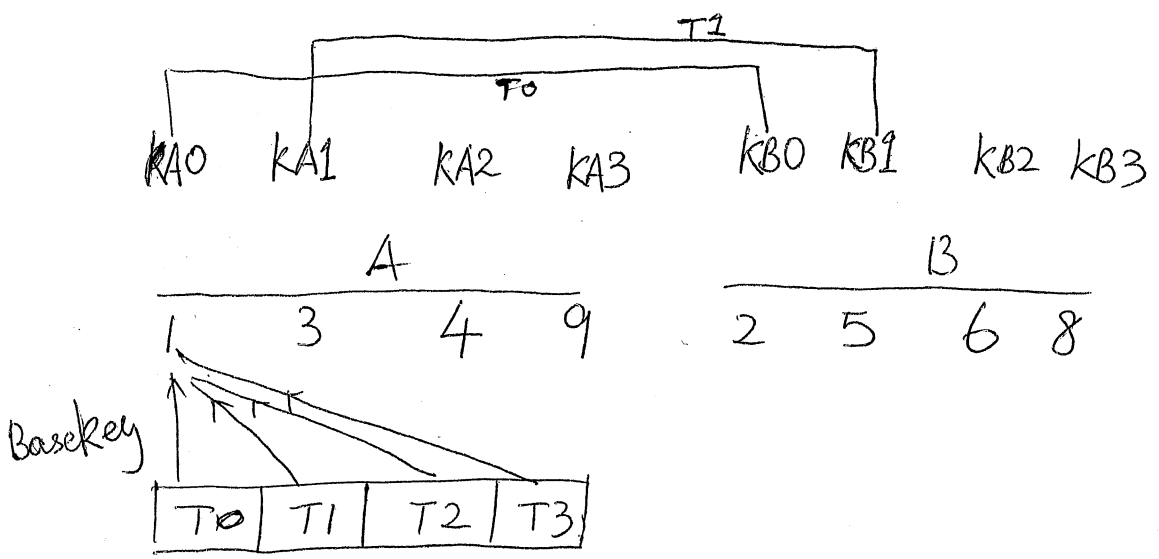
parallel merge sort (Binary search Based) [Sorting a tile]

Illustration :

② In shared memory already :







③ page:

parallel merge sort (Binary Search Based) [Sorting a tile].

Observation:

① If we merge A, B sequences into C; we know that

rank(ai, C) = i + rank(ai, B), where ai is an element in A. A and B are both sorted already.

The ingenious part is the variable lPos; which means the index i for an element ai in sorted section A. (the left sequence to be merged)

② basekey, baseVal } ⇒ the start address of the subsequences A & B, stored in shared memory, where A & B stored in contiguous memories. B is stored right after A.

{ different Thread may have different basekey value.